

CSS Interview Cheat Sheet

Complete Guide — Basics to Advanced

100+ Interview Questions & Answers

CONTENTS

- | | | |
|-----------------------|-----------------------------|----------------------------------|
| 1. Basics & Selectors | 2. Box Model | 3. Display & Visibility |
| 4. Flexbox | 5. CSS Grid | 6. Positioning |
| 7. Typography | 8. Colors & Backgrounds | 9. Transitions & Animations |
| 10. Responsive Design | 11. Pseudo-classes/elements | 12. CSS Variables |
| 13. Specificity | 14. Advanced Concepts | 15. Performance & Best Practices |

1 Basics & Selectors

Q What is CSS and what does it stand for?

CSS stands for **Cascading Style Sheets**. It is a stylesheet language used to describe the presentation/visual styling of HTML documents. "Cascading" refers to the priority scheme that determines which style rules apply when multiple rules target the same element.

Q What are the different ways to apply CSS to a webpage?

Inline: `style` attribute on element (highest specificity). **Internal:** `<style>` tag in `<head>`. **External:** separate `.css` file linked via `<link>` (recommended — best separation of concerns, caching).

Q Explain all CSS selector types with examples.

Universal: `*` — all elements

Type/Element: `p`, `div`, `h1`

Class: `.classname`

ID: `#idname` (unique per page)

Attribute: `[type="text"]`, `[href^="https"]`

Descendant: `div p` (any nested p)

Child: `div > p` (direct child only)

Adjacent sibling: `h1 + p` (immediately after)

General sibling: `h1 ~ p` (all after h1)

Q What is the difference between `class` and `id` selectors?

`id` must be unique per page, has higher specificity (100 points), used for single elements. `class` can be reused on multiple elements, has lower specificity (10 points), used for groups of elements.

Q What are attribute selectors? Give examples.

`[attr]` — has attribute | `[attr="val"]` — exact match
`[attr^="val"]` — starts with | `[attr$="val"]` — ends with
`[attr*="val"]` — contains | `[attr~="val"]` — word match

Q What is the `:is()` selector?

`:is(h1, h2, h3) span` — matches `span` inside any of the listed selectors. Reduces repetition, takes the highest specificity of its arguments.

2 Box Model

Q Explain the CSS box model.

Every element is a rectangular box made of 4 layers (inside out): **Content** → **Padding** → **Border** → **Margin**. Total width = content + padding-left + padding-right + border-left + border-right (by default).

Q What is the difference between `box-sizing: content-box` and `border-box` ?

`content-box` (default): width/height apply to content only — padding and border are added on top. `border-box`: width/height include padding and border — makes layout math much easier. Best practice: use `* { box-sizing: border-box; }` globally.

Q What is margin collapsing?

When two vertical margins meet (adjacent siblings or parent-child), they collapse into one — the larger margin wins. Doesn't happen with horizontal margins, floated elements, flex/grid items, or absolutely positioned elements.

Q What is the difference between `padding` and `margin` ?

`padding` is space inside the border (part of the element, has background). `margin` is space outside the border (transparent, between elements). Padding increases clickable area; margins create gaps between elements.

Q How do you center a block element horizontally?

Set a fixed width and use `margin: 0 auto`. For inline content: `text-align: center` on parent. Modern way: use flexbox or grid on parent.

3 Display & Visibility

Q Difference between `display: none` and `visibility: hidden` ?

`display: none` — removes element from layout flow (no space taken). `visibility: hidden` — element is invisible but still occupies space.

Q What is `display: inline` vs `block` vs `inline-block` ?

`block`: full width, new line, accepts width/height.
`inline`: only as wide as content, no width/height.
`inline-block`: flows inline but accepts width/height.

Q What does `overflow: hidden` do?

Clips content that exceeds the element's bounds. Also clears floats (creates a new block formatting context).

6 Positioning

Q Explain all CSS position values.

`static`: default, normal flow. `relative`: offset from its own position, still in flow. `absolute`: removed from flow, positioned relative to nearest non-static ancestor. `fixed`: relative to viewport. `sticky`: acts relative until scroll threshold, then fixed.

Q What is a stacking context and `z-index` ?

`z-index` controls stacking order of overlapping positioned elements. A stacking context is formed by elements with position + z-index, opacity < 1, transform, filter, etc. Children are stacked within their parent's context.

Q How does `position: sticky` work?

Acts like `relative` until the element reaches its scroll threshold (e.g. `top: 0`), then becomes `fixed` within its

Q What is `opacity: 0` vs `display: none` ?

`opacity: 0` — element is invisible, still takes space, still interactive. `display: none` — completely removed, no space, not interactive.

parent container. Parent must have a defined height.

4 Flexbox

Q What is Flexbox and when should you use it?

A one-dimensional layout model (row OR column). Best for navigation bars, aligning items, distributing space, centering, and small-scale components. Use CSS Grid for two-dimensional layouts.

Q Explain key flex container properties.

`display: flex` — activates flex context

`flex-direction`: row | column | row-reverse | column-reverse

`flex-wrap`: nowrap | wrap | wrap-reverse

`justify-content`: flex-start | flex-end | center | space-between | space-around | space-evenly

`align-items`: flex-start | flex-end | center | stretch | baseline

`align-content`: controls multi-line alignment

`gap`: spacing between items

Q Explain key flex item properties.

`flex-grow`: how much item grows relative to others (0 = no grow)

`flex-shrink`: how much item shrinks (1 = default)

`flex-basis`: initial main size before grow/shrink

`flex: 1` shorthand = grow:1, shrink:1, basis:0

`align-self`: overrides align-items for one item

`order`: changes visual order (default 0)

Q How do you center an element both horizontally and vertically with flexbox?

On parent: `display: flex; justify-content: center; align-items: center;`

Q What is the difference between `justify-content` and `align-items` ?

`justify-content` aligns items along the **main axis** (row = horizontal, column = vertical). `align-items` aligns along the **cross axis** (perpendicular to main axis).

5 CSS Grid

Q What is CSS Grid and how does it differ from Flexbox?

Grid is a **two-dimensional** layout system (rows AND columns simultaneously). Flexbox is one-dimensional. Grid is better for overall page layout; Flexbox for component-level layout. Both can be combined.

Q Explain `grid-template-columns` and the `fr` unit.

`grid-template-columns: 1fr 2fr 1fr` creates 3 columns where the middle takes twice as much space. `fr` (fractional unit) distributes available space proportionally after fixed-size columns are allocated.

```
.grid { display: grid; grid-template-columns: repeat(3, 1fr); grid-template-rows: auto 200px; gap: 16px; }
```

Q What is `grid-column` and `grid-row` span?

`grid-column: 1 / 3` — item spans from column line 1 to 3 (2 columns). `grid-column: span 2` — spans 2 columns from current position. Same applies for `grid-row`.

Q What is `grid-template-areas` ?

Allows naming grid areas for a visual layout map:

```
grid-template-areas: "header header" "sidebar main" "footer footer";
/* Then assign: */ header { grid-area: header; }
```

Q What does `minmax()` do in grid?

`minmax(min, max)` sets a size range. Example: `grid-template-columns: repeat(auto-fill, minmax(200px, 1fr))` creates as many 200px+ columns as fit — responsive without media queries.

Q Difference between `auto-fill` and `auto-fit` ?

`auto-fill`: creates as many columns as fit, including empty ones. `auto-fit`: creates columns as fit, but collapses empty ones — items expand to fill remaining space.

7 Typography

Q What are CSS units? Which to use when?

`px`: fixed pixels — precise but not scalable
`em`: relative to parent font-size
`rem`: relative to root (html) font-size — preferred
`%`: relative to parent
`vw/vh`: % of viewport width/height
`ch`: width of "0" character — good for max-width
 Use `rem` for font, `px` for borders, `%/fr` for layout.

Q What is `line-height` and best practices?

Sets the height of a line of text. Use unitless values like `1.5` or `1.6` (relative to font-size) — avoids inheritance issues. Avoid fixed px values.

Q Difference between `font-weight` values?

Numeric (100–900, multiples of 100) or keyword (`normal` = 400, `bold` = 700). Value only works if the font file for that weight is loaded.

Q What is `text-overflow: ellipsis` ?

Clips overflowing text with "...". Requires: `white-space: nowrap; overflow: hidden; text-overflow: ellipsis;` — all three needed.

8 Colors & Backgrounds

Q What color formats does CSS support?

`keyword`: red, blue, transparent
`hex`: `#rrggbb`, `#rgb`, `#rrggbbaa`
`rgb()`: `rgb(255, 0, 0)`
`rgba()`: with alpha channel
`hsl()`: hue(0–360), saturation%, lightness%
`hsla()`: with alpha
`oklch()`: modern perceptual color space

Q How does `background-size` work?

`cover`: scales to cover entire area (may crop). `contain`: scales to fit inside (may have gaps). Specific values: `100px`, `200px` or `50%`.

Q How do you create a CSS gradient?

`background: linear-gradient(to right, #f00, #00f)` — direction then color stops. `radial-gradient(circle, #f00, #00f)`. Can use multiple color stops and percentages.

Q What is `background-clip` ?

Controls how far background extends: `border-box` (default), `padding-box`, `content-box`, `text` (clips to text shape — combine with `-webkit-background-clip: text`).

9 Transitions & Animations

Q What is the difference between `transition` and `animation` ?

`transition`: smooth change between two states (triggered by user action like hover). `animation`: defines keyframes for complex multi-step animations that can run automatically and loop.

Q Explain CSS transition syntax.

`transition: property duration timing-function delay`

Example: `transition: all 0.3s ease-in-out 0s`. Multiple: comma-separated. Use `transform` and `opacity` for best performance (GPU-composited).

Q Explain `@keyframes` and `animation` properties.

```
@keyframes fadeIn { 0% { opacity: 0; transform: translateY(-10px); } 100% { opacity: 1; transform: translateY(0); } } .el { animation: fadeIn 0.5s ease forwards; }
```

`animation` shorthand: name duration timing-function delay iteration-count direction fill-mode play-state.

Q What are easing functions / timing functions?

`ease` (default), `linear`, `ease-in`, `ease-out`, `ease-in-out`, `cubic-bezier(x1,y1,x2,y2)` for custom curves, `steps(n)` for discrete jumps.

Q Which CSS properties are most performant to animate and why?

`transform` and `opacity` — they run on the GPU compositor thread without triggering layout or paint. Avoid animating `width`, `height`, `margin`, `top/left` — these trigger layout reflow.

Q What does `will-change` do?

Hints to the browser to pre-optimize for an upcoming change: `will-change: transform`. Promotes element to its own compositor layer. Use sparingly — it consumes memory.

10 Responsive Design & Media Queries

Q What is responsive web design?

Design approach that makes web pages look and function well across all screen sizes using fluid layouts, flexible images, and CSS media queries.

Q Explain CSS media queries with examples.

```
/* Mobile first approach */ @media (min-width: 768px) { ... } /* tablet */ @media (min-width: 1024px) { ... } /* desktop */ @media (prefers-color-scheme: dark) { ... } @media (prefers-reduced-motion: reduce) { ... } @media print { ... }
```

Q What is mobile-first design?

Writing base CSS for mobile, then adding `min-width` media queries to enhance for larger screens. Recommended approach — ensures performance on mobile and progressive enhancement on desktop.

Q What is the viewport meta tag and why is it needed?

`<meta name="viewport" content="width=device-width, initial-scale=1">` — tells mobile browsers not to scale down the page to fit the screen. Without it, media queries won't work correctly on mobile.

Q What are CSS container queries?

Like media queries but based on the container's size rather than viewport: `@container (min-width: 400px) { ... }`. Requires `container-type: inline-size` on parent. Better for reusable component-based design.

Q What is `clamp()` in CSS?

`clamp(min, preferred, max)` — responsive value within bounds. Example: `font-size: clamp(1rem, 2.5vw, 2rem)` — scales with viewport, never below 1rem or above 2rem. Eliminates many media queries.

11 Pseudo-classes & Pseudo-elements

Q What is the difference between pseudo-class and pseudo-element?

Pseudo-class (`:`) selects elements in a specific state: `:hover`, `:focus`, `:nth-child()`, `:not()`. **Pseudo-element** (`::`) targets a part of an element: `::before`, `::after`, `::first-line`, `::placeholder`.

Q What are `::before` and `::after` ?

Insert generated content before/after element's content. Require `content` property (can be `""`). Commonly used for decorative elements, icons, and CSS tricks without extra HTML.

Q Explain `:nth-child()` vs `:nth-of-type()` .

`:nth-child(n)` counts all sibling elements. `:nth-of-type(n)` counts only siblings of the same element type. Pattern: `2n` = even, `2n+1` = odd, `3n` = every 3rd.

Q What is `:focus-visible` vs `:focus` ?

`:focus` triggers on any focus (including mouse click). `:focus-visible` only shows focus indicator when navigating by keyboard — better UX: hides the focus ring for mouse users, shows it for keyboard navigation.

Q Name some important pseudo-classes for forms.

`:valid` / `:invalid` — validation state. `:required` / `:optional`. `:checked` — checkbox/radio state. `:disabled` / `:enabled`. `:placeholder-shown` — placeholder visible.

12 CSS Variables (Custom Properties)

Q What are CSS custom properties (variables)?

Reusable values defined with `--name: value` syntax, accessed with `var(--name)`. Defined in `:root` for global scope. Unlike preprocessor variables, they are dynamic and can change at runtime.

```
:root { --color-primary: #2563eb; --spacing-md: 16px; }
.btn { background: var(--color-primary); padding: var(--spacing-md); }
```

Q What is a fallback value in `var()` ?

`var(--color, #333)` — uses `#333` if `--color` is not defined. Second argument is the fallback.

Q How are CSS variables different from SASS/LESS variables?

13 Specificity & Cascade

Q How is CSS specificity calculated?

Specificity is a 4-part score (a, b, c, d):

a: inline styles (1000)

b: IDs (100 each)

c: classes, attributes, pseudo-classes (10 each)

d: elements, pseudo-elements (1 each)

Higher score wins. `!important` overrides all (avoid it).

Q What is the CSS cascade?

The algorithm that determines which CSS rule applies when multiple rules target the same element. Order: **Origin & importance** → **Specificity** → **Source order** (later wins).

Q What is CSS inheritance?

Some properties (like `color`, `font-family`, `font-size`) are inherited by children by default. Others (like

CSS variables live in the DOM, cascade, are scoped, and can be changed with JavaScript. SASS/LESS variables are compiled away — they're static and don't exist at runtime.

`margin`, `padding`, `border`) are not. Use `inherit`, `initial`, `unset`, `revert` keywords to control.

Q What is `:where()` selector and how does `:is()` the it differ from

Both match the same elements, but `:where()` always has **zero specificity** — useful for base styles that are easy to override. `:is()` uses the highest specificity of its arguments.

14 Advanced CSS Concepts

Q What is a CSS preprocessor? Name some.

A tool that extends CSS with features like variables, nesting, mixins, functions, and imports, then compiles to standard CSS. Examples: **SASS/SCSS** (most popular), **LESS**, **Stylus**.

Q What is CSS-in-JS?

Writing CSS within JavaScript files, scoped to components. Examples: **styled-components**, **Emotion**, **CSS Modules**. Advantages: scoped styles, dynamic styling, co-location. Disadvantages: runtime overhead, bundle size.

Q What is a BEM methodology?

Block Element Modifier naming convention. Block: `.card`, Element: `.card_title`, Modifier: `.card--featured`. Provides clear, predictable class names and avoids specificity conflicts.

Q What is the `transform` property?

Applies 2D/3D transformations without affecting document flow: `translate(x,y)`, `scale(x,y)`, `rotate(deg)`, `skew(x,y)`. Multiple transforms: space-separated. Does not trigger layout reflow — GPU accelerated.

Q What is `clip-path` ?

Creates a clipping region to show only part of an element. Shapes: `circle()`, `ellipse()`, `polygon()`, `inset()`, `path()`. Animatable. Example: `clip-path: polygon(50% 0%, 0% 100%, 100% 100%)` creates a triangle.

Q What is the `filter` property?

Applies graphical effects: `blur(px)`, `brightness(%)`, `contrast(%)`, `grayscale(%)`, `saturate(%)`, `hue-rotate(deg)`, `drop-shadow()`. Creates a stacking context. Multiple filters: space-separated.

Q What is `backdrop-filter` ?

Like `filter` but applies to the content behind an element. Used for glassmorphism effects: `backdrop-filter: blur(10px)`. Element must have a semi-transparent background.

Q What are CSS logical properties?

Writing-mode-aware alternatives: `margin-inline-start` instead of `margin-left`, `padding-block` instead of `padding-top/bottom`. Better for internationalization (RTL languages).

Q What is `aspect-ratio` ?

`aspect-ratio: 16 / 9` maintains a fixed ratio as the element resizes. Replaces old padding-top hack for responsive iframes and images.

Q What is the difference between `:root` and `html` ?

Both select the root element, but `:root` has higher specificity (pseudo-class = 10 points vs 1 for element). Also, `:root` works in non-HTML documents (SVG, XML). Commonly used for CSS variable definitions.

15 Performance & Best Practices

Q What triggers layout reflow vs repaint vs compositing?

Reflow (most expensive): changing geometry — width, height, margin, padding, top, left, font-size

Repaint: visual changes only — color, background, visibility

Compositing (cheapest): transform, opacity — GPU handles it

Q What is CSS containment and the `contain` property?

`contain: layout paint style` — isolates element subtree from the rest of the document. Browser can skip recalculating outside the boundary. Improves performance for independent components.

Q What is critical CSS?

The minimal CSS needed to render above-the-fold content. Inlined in `<head>` to eliminate render-blocking. Remaining CSS loaded asynchronously. Improves First Contentful Paint (FCP).

Q What is the `@layer` CSS rule?

Cascade layers allow organizing CSS into named layers with explicit priority. Styles in later layers win within the same importance level. Useful for managing third-party CSS, resets, and utilities without specificity wars.

Q How do you avoid common CSS pitfalls?

Avoid `!important` except for utilities. Use `box-sizing: border-box` globally. Use relative units (rem) over px for fonts. Prefer CSS variables over repeated magic numbers. Keep selectors short. Use `will-change` sparingly. Write mobile-first.

Q What are some modern CSS features to know in 2024+?

Container Queries

:has() selector

CSS Nesting

@layer

subgrid

oklch()

color-mix()

@starting-style

view transitions

anchor positioning

scroll-driven animations

Q What is the `:has()` relational pseudo-class?

Selects a parent/ancestor based on its children: `form:has(:invalid)` selects a form that contains an invalid input. Known as the "parent selector". Fully supported in modern browsers.

Q What is CSS nesting?

Native CSS now supports nesting without preprocessors:

```
.card { color: black; &:hover { background: #f0f0f0; } & .title { font-size: 1.2rem; } }
```